

SUMMER INTERNSHIP - 2026

FOUNDATIONS OF DATA ANALYTICS WITH PYTHON

1. CURRICULUM (8 WEEKS / 40 DAYS)

Duration & Schedule -

- **Total Duration:** 40 Days (8 Weeks)
- **Theory Session:** 9:30 AM - 11:30 AM (2 hours)
- **Practical Session:** 12:00 PM - 2:00 PM (3 hours)
- **Venue:** C Block
- **Instructor:** Mr. Maninder Singh

WEEK-WISE PLAN -

Week 1: Python Core - Fundamentals, Data Structures & Functions

Topics Covered:

- Python Basics: Variables, data types (strings, integers, floats, booleans), operators (arithmetic, relational, logical)
- Control Structures: if-else statements, nested conditions, elif chains
- Loops: for loops, while loops, break, continue, nested loops
- Tools & Setup: Installation of Anaconda, Jupyter Notebook orientation, first programs
- Lists & Tuples: Creation, indexing, slicing, built-in methods, immutability, packing & unpacking
- Dictionaries & Sets: Key-value pairs, nested dicts, iteration, set operations (union, intersection, difference)
- Functions: def, return, default arguments, *args, **kwargs, scope rules
- Lambda & List Comprehensions: Concise one-liner expressions for data manipulation
- Introduction to Libraries: Overview of NumPy, Pandas, Matplotlib

Day-wise Plan:

Day	Theory (2 hrs)	Practical (3 hrs)
Day 1	Python syntax, variables, all data types; operators — arithmetic, relational, logical, assignment; type casting; control structures: if, elif, else, nested conditions	Write first Python programs in Jupyter; build a temperature & currency converter; create a grade classifier using if-elif-else; solve type-casting exercises
Day 2	Loops — for, while, break, continue, nested loops; loop patterns and iteration techniques; functions: def, return, scope, default arguments, *args, **kwargs	Print number patterns and factorial using loops; write a multiplication table generator; build reusable math utility functions (area, power, average)
Day 3	Lists — creation, indexing, slicing, append, remove, sort, reverse; tuples — immutability, packing & unpacking; sets — unique values, union, intersection, difference, symmetric difference	List manipulation tasks: sort, filter, reverse, slice a marks list; perform set operations on two student enrollment lists; unpack tuples in a loop to display student info
Day 4	Dictionaries — creation, key-value access, nested dicts, iteration (items(), keys(), values());	Build a student record dictionary: add, delete, search, display; use list comprehensions to filter students scoring above 75; rewrite loop-based

Day	Theory (2 hrs)	Practical (3 hrs)
	lambda functions; list comprehensions; dict comprehensions; filter() and map()	transformations as one-liners using lambda & comprehensions
Day 5	Anaconda & Jupyter setup — environments, kernels, markdown cells; introduction to NumPy, Pandas, Matplotlib: what they do & why; end-to-end mini-project integration session	Install and verify all libraries; run import checks; build the combined mini-project: Temperature Converter + Grade Classifier + Student Record Manager in a single polished Jupyter Notebook

Mini Project: Temperature Converter, Grade Classifier & Student Record Manager

- Section 1 — Temperature Converter: Accept Celsius input and convert to Fahrenheit and Kelvin using functions
- Section 2 — Grade Classifier: Accept marks for 5 subjects, compute average, classify grade using if-elif-else
- Section 3 — Student Record Manager: Store records in a list of dicts; add, delete, search, display using functions and list comprehensions
- All three sections combined in a single formatted Jupyter Notebook with markdown headings and output cells

Outcome: Proficiency in Python fundamentals, data structures, and functional programming in 5 days

Week 2: NumPy for Numerical Computing

Topics Covered:

- NumPy Arrays: Creation, array properties — shape, dtype, ndim, size
- Array Operations: Arithmetic, broadcasting, vectorization vs. loops
- Reshaping & Indexing: reshape(), flatten(), transpose(), fancy indexing
- Slicing: 1D and 2D array slicing; boolean masking
- Mathematical Functions: np.mean, np.std, np.sum, np.min, np.max, np.percentile
- Date and Time: datetime module, formatting dates, calculating time differences

Day-wise Plan:

Day	Theory (2 hrs)	Practical (3 hrs)
Day 6	NumPy arrays — creation (zeros, ones, arange, linspace, random), shape, dtype, ndim, size; difference between lists and arrays	Create 1D & 2D arrays; compare list vs array arithmetic speed; explore array properties; create identity and random matrices
Day 7	Array operations — element-wise arithmetic, broadcasting rules, vectorized operations; reshaping: reshape(), flatten(), transpose()	Vectorized arithmetic on a sales figures array; reshape a 1D marks array into a 2D subject-student matrix; transpose and flatten practice
Day 8	Indexing & slicing 1D and 2D arrays; fancy indexing with integer arrays; boolean masking and conditional selection	Slice rows and columns from a 2D student marks array; use boolean mask to select students scoring above average; extract every 2nd element
Day 9	NumPy statistical functions: mean, median, std, var, min, max, percentile, cumsum; axis-wise operations on 2D arrays	Compute subject-wise and student-wise statistics; find class topper using argmax(); compute running totals with cumsum()
Day 10	Date & time with datetime module: date, time, timedelta, strptime, strftime; calculating age, days between dates; combining with arrays	Calculate each student's age from a DOB list; find number of days until exam; format and display dates; build a date-range array for a semester

Mini Project: Marks Analysis with NumPy

- Load a 2D NumPy array of student marks (5 students × 5 subjects)
- Compute subject-wise mean, std, min, max and student-wise total and average
- Identify top performers using boolean masking and `argmax()`
- Display a formatted summary report inside a Jupyter Notebook

Outcome: *Efficient numerical data manipulation and statistical computation using NumPy*

Week 3: Pandas - Data Import & Cleaning

Topics Covered:

- Pandas Series & DataFrames: Creation, properties, index structure
- Reading Data: CSV, Excel, JSON, and text files with `pd.read_*`()
- Indexing & Slicing: `loc[]`, `iloc[]`, boolean filtering, chained conditions
- Handling Missing Values: `isnull()`, `fillna()`, `dropna()`, `interpolate()`
- Removing Duplicates & Outliers: `drop_duplicates()`, `value_counts()`
- Data Type Conversion: `astype()`, column renaming, sorting, `reset_index()`

Day-wise Plan:

Day	Theory (2 hrs)	Practical (3 hrs)
Day 11	Pandas Series & DataFrames — creation from dicts, lists, arrays; index, columns, dtypes, shape, <code>info()</code> , <code>describe()</code>	Create DataFrames manually and from dictionaries; explore a sample DataFrame using <code>head()</code> , <code>tail()</code> , <code>info()</code> , <code>describe()</code> , shape
Day 12	Reading data — <code>pd.read_csv()</code> , <code>pd.read_excel()</code> , <code>pd.read_json()</code> ; parameters: header, index_col, usecols, nrows, sep; writing back with <code>to_csv()</code>	Import a real-world sales CSV; inspect with <code>info()</code> ; load an Excel file; export a cleaned subset back to CSV
Day 13	Indexing with <code>loc[]</code> (label-based), <code>iloc[]</code> (position-based); boolean filtering with single and multiple conditions using <code>&</code> and <code> </code> ; <code>.query()</code> method	Filter rows where sales > 5000; select students with grade A in Science; chain multiple conditions to extract a specific customer segment
Day 14	Handling missing values — <code>isnull()</code> , <code>sum()</code> , <code>fillna()</code> with mean/median/mode/constant, <code>dropna()</code> with thresh; <code>interpolate()</code> for numeric columns	Identify missing value counts per column; fill numeric nulls with column mean; drop rows missing more than 2 values; apply interpolation on a time column
Day 15	Removing duplicates — <code>drop_duplicates()</code> , keep parameter; <code>astype()</code> for type conversion; column rename; <code>sort_values()</code> ; <code>reset_index()</code> ; <code>value_counts()</code>	Remove duplicate order entries; fix mixed-type price column using <code>astype()</code> ; rename columns to snake_case; sort by date and reset index

Mini Project: Sales Dataset Cleaning Pipeline

- Load a messy sales CSV dataset containing missing values, duplicates, and wrong data types
- Identify and report all data quality issues using `isnull()`, `duplicated()`, and `dtypes`
- Apply cleaning steps: fill/drop nulls, remove duplicates, fix types, rename columns, sort
- Save the cleaned and validated dataset back as a new CSV file with a quality summary

Outcome: *Ability to import, inspect, and clean real-world datasets using Pandas*

Week 4: Pandas - Data Manipulation & Transformation

Topics Covered:

- GroupBy Operations: `groupby()`, `agg()`, `transform()`, named aggregations
- Merging & Joining: `merge()` with inner/outer/left/right joins, `concat()`, `join()`
- Apply & Map: `apply()` with custom functions, `map()`, `applymap()` / `map()` on DataFrames
- String Operations: str accessor — `split`, `strip`, `upper`, `lower`, `contains`, `replace`, `extract`
- Pivot Tables & Cross-Tabs: `pd.pivot_table()`, `pd.crosstab()`
- Practical Application: End-to-end manipulation pipeline on a real business dataset

Day-wise Plan:

Day	Theory (2 hrs)	Practical (3 hrs)
Day 16	GroupBy — <code>groupby()</code> , <code>agg()</code> with multiple functions, named aggregations, <code>transform()</code> for group-wise normalisation, <code>size()</code> vs <code>count()</code>	Group sales data by region and product category; compute total, mean, and max revenue; use <code>transform()</code> to add a column showing each row's % of group total
Day 17	Merging DataFrames — <code>merge()</code> with how parameter (inner, left, right, outer), on vs left_on/right_on; <code>concat()</code> for stacking; <code>join()</code> on index	Merge orders and products DataFrames on <code>product_id</code> ; outer-join customers to flag unmatched; stack quarterly sales sheets with <code>concat()</code>
Day 18	<code>apply()</code> with lambda and custom functions on rows and columns; <code>map()</code> for value replacement; <code>applymap()/map()</code> for element-wise ops; handling errors in <code>apply()</code>	Apply a discount function across price column; map grade codes to labels; compute a derived "profit margin" column using a custom <code>apply</code> function
Day 19	String operations using str accessor — <code>strip()</code> , <code>upper()</code> , <code>lower()</code> , <code>split()</code> , <code>contains()</code> , <code>replace()</code> , <code>extract()</code> with regex; handling NaN in string columns	Extract first and last name from <code>full_name</code> column; clean and validate email addresses; flag rows where product name contains "expired"
Day 20	Pivot tables — <code>pd.pivot_table()</code> with values, index, columns, <code>aggfunc</code> , <code>fill_value</code> , margins; <code>pd.crosstab()</code> for frequency tables; sorting and styling pivot output	Build a pivot table of monthly sales by product category; use <code>crosstab</code> to show order count by region and status; add grand totals using <code>margins=True</code>

Mini Project: E-Commerce Data Analysis

- Merge orders, customers, and products datasets on common keys using appropriate join types
- Group by category and month to compute revenue, order count, and average order value
- Apply string cleaning on customer name and email columns
- Build a pivot table showing cross-tabular monthly revenue by product category

Outcome: Advanced data wrangling and transformation using Pandas

Week 5: Exploratory Data Analysis (EDA) & Statistics

Topics Covered:

- Descriptive Statistics: Mean, median, mode, variance, standard deviation, range
- Distribution Analysis: Skewness, kurtosis, percentiles, quartiles, IQR

- Correlation & Covariance: `corr()`, `cov()`, Pearson vs Spearman, interpretation
- Outlier Detection: IQR fence method and Z-score identification
- Matplotlib Basics: Figure, axes, line plots, bar charts, histograms, formatting
- EDA Workflow: Combining statistics and visualization for insight extraction

Day-wise Plan:

Day	Theory (2 hrs)	Practical (3 hrs)
Day 21	Descriptive statistics — mean, median, mode, variance, std, range; <code>describe()</code> ; skewness and kurtosis interpretation; when to use mean vs median	Compute full stats summary on a student performance dataset; compare mean vs median for a skewed column; identify which subjects have highest variance
Day 22	Distribution analysis — histograms, frequency tables; percentiles and quartiles with <code>np.percentile()</code> ; IQR calculation; normal vs skewed distributions	Plot histograms for marks and study hours; compute Q1, Q3, IQR; check if data is normally distributed; compare distributions across two groups
Day 23	Correlation — <code>corr()</code> Pearson and Spearman; <code>cov()</code> ; interpreting correlation coefficients; multicollinearity basics; heatmap preview	Compute correlation matrix for student dataset; identify pairs with strong positive/negative correlation; discuss which features to use in analysis
Day 24	Outlier detection — IQR fence ($Q1 - 1.5 \times IQR$, $Q3 + 1.5 \times IQR$); Z-score method ($ Z > 3$); flagging vs dropping outliers; impact on statistics	Detect outliers in marks and study hours columns using both methods; flag them in a new column; compare statistics before and after outlier removal
Day 25	Matplotlib — Figure and Axes architecture; line plots, bar charts, histograms; labels, titles, legends, grid, colors, linestyle; <code>plt.subplots()</code> basics; saving figures	Create line plot of monthly sales trend; bar chart of subject-wise average marks; histogram of score distribution; add proper labels and save as PNG

Mini Project: Student Performance EDA

- Load student marks dataset and compute a complete descriptive statistics summary
- Identify correlations between study hours, attendance, and final grades
- Detect and flag outlier students using IQR and Z-score methods
- Visualize distributions, correlations, and subject comparisons using Matplotlib

Outcome: Ability to explore, statistically analyze, and visualize real-world datasets

Week 6: Advanced Visualization - Matplotlib & Seaborn

Topics Covered:

- Matplotlib Advanced: Subplots with `GridSpec`, twin axes (`twinx()`), annotations, style sheets, `savefig()` with DPI
- Scatter & Bubble Charts: Multi-variable relationship visualization
- Box Plots & Violin Plots: Distribution comparison across groups
- Seaborn Fundamentals: `sns.set_theme()`, `sns.set_palette()`, figure-level vs axes-level functions
- Seaborn Statistical Plots: `histplot()`, `kdeplot()`, `boxplot()`, `violinplot()`, `countplot()`
- Advanced Seaborn: `heatmap()`, `pairplot()`, `FacetGrid()`, `regplot()`, `Implot()`

Day-wise Plan:

Day	Theory (2 hrs)	Practical (3 hrs)
Day 26	Advanced Matplotlib — GridSpec for complex layouts, <code>twinx()</code> dual-axis charts, <code>axhline/axvline</code> reference lines, annotations with <code>annotate()</code> , <code>plt.style.use()</code> themes	Build a dual-axis chart overlaying sales volume (bar) and revenue (line); annotate peak and lowest data points; apply <code>seaborn-v0_8-darkgrid</code> style; export as 300-DPI PNG
Day 27	Scatter plots — <code>plt.scatter()</code> with color and size encoding; bubble charts; pair plots with <code>pd.plotting.scatter_matrix()</code> ; reading multi-variable patterns	Plot scatter of study hours vs marks colored by grade; create a bubble chart where bubble size = attendance %; build a scatter matrix for the sales dataset
Day 28	Box plots — <code>plt.boxplot()</code> , grouped box plots; violin plots — <code>plt.violinplot()</code> ; reading quartiles, median, whiskers, outlier dots; comparing distributions	Build grouped box plots comparing subject-wise score distributions; plot violin plots for marks by gender; overlay individual data points using <code>stripplot</code> preview
Day 29	Seaborn introduction — relationship with Matplotlib; <code>sns.set_theme()</code> , <code>set_palette()</code> , <code>set_context()</code> ; axes-level: <code>histplot()</code> , <code>kdeplot()</code> , <code>boxplot()</code> , <code>violinplot()</code> , <code>countplot()</code>	Apply Seaborn themes to restyle previous charts; plot KDE + histogram overlay on marks data; grouped boxplot and violin plot with <code>hue=</code> for gender comparison
Day 30	Advanced Seaborn — <code>sns.heatmap()</code> with <code>annot</code> , <code>fmt</code> , <code>cmap</code> ; <code>sns.pairplot()</code> with <code>hue</code> ; <code>sns.FacetGrid()</code> for multi-panel category plots; <code>sns.regplot()</code> and <code>lmlplot()</code> for regression lines; combining with Matplotlib for final tweaks	Generate annotated correlation heatmap; <code>pairplot</code> with <code>hue=grade</code> ; <code>FacetGrid</code> of score distributions per subject; <code>regplot()</code> for study hours vs marks; final <code>savefig()</code> with title

Mini Project: Sales & Student Data Visualization Dashboard

- Build a styled multi-panel Matplotlib dashboard (2x2 grid) — line, bar, scatter, and box plots
- Apply a consistent Seaborn theme and custom palette across all charts
- Generate an annotated correlation heatmap and pairplot for the sales dataset
- Create a `FacetGrid` of category-wise revenue distributions; export final dashboard as high-DPI PNG

Outcome: Ability to create publication-quality static and statistical visualizations using Matplotlib and Seaborn

Week 7: Case Study - Weather Dataset Time-Series Analysis

Topics Covered:

- Dataset Exploration: Load and explore a multi-year weather dataset
- Time-Series Handling: Parse date columns, set `datetime` index, `resample()`
- Rolling Statistics: `rolling()`, `expanding()`, moving averages
- Temperature & Rainfall Trends: Monthly and seasonal aggregations
- Multi-Variable Analysis: Humidity, pressure, wind speed patterns
- Report Building: Comprehensive annotated Jupyter Notebook report

Day-wise Plan:

Day	Theory (2 hrs)	Practical (3 hrs)
Day 31	Time-series fundamentals — <code>pd.to_datetime()</code> , <code>set_index()</code> , <code>DatetimeIndex</code> properties (<code>.year</code> , <code>.month</code> , <code>.day_name()</code>); <code>resample()</code> with M, Q, A frequencies; resampling aggregations	Load weather CSV; parse and set date as index; extract year, month, day-of-week columns; resample to monthly averages; verify data continuity
Day 32	Rolling statistics — <code>rolling(window).mean/std/min/max</code> ; <code>expanding</code>	Compute 7-day and 30-day rolling average temperatures; overlay rolling mean on daily

Day	Theory (2 hrs)	Practical (3 hrs)
	windows; ewm() for exponential moving average; detecting trends vs noise	data plot; compare simple vs exponential moving averages
Day 33	Seasonal analysis — groupby month and season; monthly min/max/avg temperature; detecting seasonal peaks and troughs; year-over-year comparison	Plot monthly average temperature for each year as separate lines; compute season-wise (Summer/Monsoon/Winter) statistics; identify hottest and coldest months
Day 34	Multi-variable analysis — rainfall, humidity, wind speed patterns; correlation between weather variables; time-lagged correlations; handling irregular data	Plot rainfall bar chart by month; scatter humidity vs temperature; compute correlation matrix for all weather variables; check if rainfall follows temperature peaks
Day 35	Report building — structuring a Jupyter Notebook as a professional report: markdown narrative, chart annotations, summary tables, conclusions; exporting to HTML	Compile all charts (6+ types) into a single notebook with markdown narrative; add annotated key findings on each chart; export notebook as HTML report

Mini Project: Weather Data Analysis Report

- Load and clean a multi-year weather dataset; parse dates and set datetime index
- Compute monthly, seasonal, and rolling statistics for temperature, rainfall, and humidity
- Build a comprehensive visual report with 6+ annotated chart types
- Export the final Jupyter Notebook as a structured HTML report with narrative insights

Outcome: End-to-end data analysis and reporting workflow on real-world time-series data

Week 8: Mini-Project, Group Work & Final Presentation

Topics Covered:

- Mini-Project Development: Apply full EDA pipeline on a chosen real-world dataset
- Peer Collaboration: Group-based data storytelling and insight narration
- Advanced Review: Recap of key EDA, visualization, and Pandas techniques
- Report Writing: Structuring a professional Jupyter Notebook report
- Project Presentation: Present findings with visualizations to the class
- Evaluation & Feedback: Q&A session, peer review scoring, and instructor grading

Day-wise Plan:

Day	Theory (2 hrs)	Practical (3 hrs)
Day 36	Project briefing — domain selection, dataset sourcing, project scope; EDA planning checklist; how to structure a data analytics report; team roles and responsibilities	Teams finalize domain and dataset; complete data loading, initial exploration (shape, dtypes, nulls, describe()); define 3 key analytical questions to answer
Day 37	Recap session — Pandas cleaning & transformation best practices; EDA workflow review; common mistakes and how to avoid them; visualization selection guide (which chart for which data)	Complete data cleaning pipeline; perform full EDA: stats, correlations, outlier detection; draft at least 4 charts answering the analytical questions defined on Day 36
Day 38	Advanced review — GroupBy patterns, multi-index operations, time-series if applicable; code optimization and readability (PEP8 basics, meaningful variable names, comments)	Peer code review session: swap notebooks with another team and give written feedback; refactor code based on feedback; finalize all 6+ visualizations

Day	Theory (2 hrs)	Practical (3 hrs)
Day 39	Report structuring — Jupyter Notebook as a report: introduction, data description, EDA findings, visualizations, conclusions, limitations; markdown formatting best practices	Write narrative markdown for each section; annotate every chart with 1–2 sentence insight; build summary statistics table; prepare a 10-slide presentation deck
Day 40	Final presentation day — each team presents for 10 minutes followed by 5-minute Q&A; peer scoring rubric; instructor evaluation; certificate eligibility review	Present findings to instructor and peers; answer Q&A; complete peer review scoring forms; receive feedback; submit final Jupyter Notebook and slide deck

Mini Project: Group Mini-Project - Real-World Dataset (Domain of Choice)

- Teams of 3–4 students choose one domain: e-commerce, healthcare, climate, or sports
- Apply the full analytics pipeline: import → clean → EDA → visualize → narrate insights
- Use at least 6 different visualization types with Seaborn and Matplotlib
- Submit a polished Jupyter Notebook report and present with a 10-slide deck

Outcome: *Ability to independently complete and present an end-to-end data analytics project*

Table 1: Week-wise Curriculum Plan with Topics, Mini Projects, and Learning Outcomes

Week	Topics Covered	Mini Project	Learning Outcome
Week 1	Python basics, operators, control flow, loops, Anaconda setup, lists, tuples, dicts, sets, functions, lambda & comprehensions	Temperature Converter, Grade Classifier & Student Record Manager	Python fundamentals, data structures & functions
Week 2	NumPy arrays, operations, reshaping, indexing, statistical functions, date & time	Marks Analysis with NumPy	Numerical computing with NumPy
Week 3	Pandas Series/DataFrames, data import (CSV/Excel/JSON), cleaning, missing values, duplicates	Sales Dataset Cleaning Pipeline	Data import & cleaning pipeline
Week 4	GroupBy, merge/join, apply, string operations, pivot tables, crosstab	E-Commerce Data Analysis	Advanced data manipulation
Week 5	Descriptive stats, distributions, correlation, outlier detection (IQR & Z-score), Matplotlib basics	Student Performance EDA	Exploratory data analysis
Week 6	Advanced Matplotlib (GridSpec, twinx, annotations), scatter/box/violin, Seaborn themes, heatmap, pairplot, FacetGrid, regplot	Sales & Student Data Visualization Dashboard	Publication-quality data visualization
Week 7	Time-series parsing, resample, rolling averages, seasonal trends, multi-variable analysis, report building	Weather Data Analysis Report	Real-world time-series EDA & reporting
Week 8	Mini-project development, group collaboration, code review, report writing, presentation, peer evaluation	Group Mini-Project (chosen domain)	End-to-end analytics project

2. CERTIFICATION DETAILS

Students are required to complete the following certifications during the internship:

Python Certification

<https://www.sanfoundry.com/certification/python-certification/>

Internship Completion Certificate -

Issued by **Geeta Technical Hub** upon successful completion of the internship.

Criteria

- Completion of all three online certifications
- Submission of certification screenshots or PDFs
- Completion of all weekly mini-projects
- Submission and presentation of the final group mini-project
- Minimum 80% attendance across all 40 days

3. PROJECTS

Project 1: Weather Dataset Analysis (Individual)

Features:

- Load and clean a multi-year real-world weather dataset in CSV format
- Parse date columns and set up a proper time-series index
- Compute monthly and seasonal averages for temperature, rainfall, and humidity
- Detect outlier weather events using the IQR or Z-score method
- Build a comprehensive visual report with a minimum of 6 chart types
- Annotate charts with key insights and trend observations

Project 2: Student Performance Dashboard (Individual)

Features:

- Import a student marks dataset containing multiple subjects
- Clean the data: handle missing entries and correct data types
- Perform full EDA: descriptive statistics, correlations, distributions
- Identify top, average, and low performers using filtering conditions
- Visualize subject-wise performance with bar, box, and violin plots using Seaborn
- Export a final polished Jupyter Notebook report with all charts and narrative

Project 3: Group Mini-Project - Domain of Choice (Group)

Features:

- Teams of 3–4 students select one domain: e-commerce, healthcare, climate, or sports
 - Apply the full analytics pipeline: import, clean, EDA, visualize, and derive insights
 - Use at least 6 different visualization types using Matplotlib and Seaborn
 - Write narrative-driven insights within a structured Jupyter Notebook
 - Present findings to the class with a 10-minute slide deck presentation
 - Participate in Q&A and peer review evaluation session
-